

PRACTICAL-01

AIM: Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

1.) BUBBLE SORT

PROGRAM:

```
#include <iostream>

using namespace std;

class bubblesort {
public:
    int arr[100], size, i, j, temp; void
    getinput() { cout << "enter the size of
    the array:"; cin >> size; cout << "enter
    the elements:";

    for (i = 0; i < size; i++) {
        cin >> arr[i];
    }
}

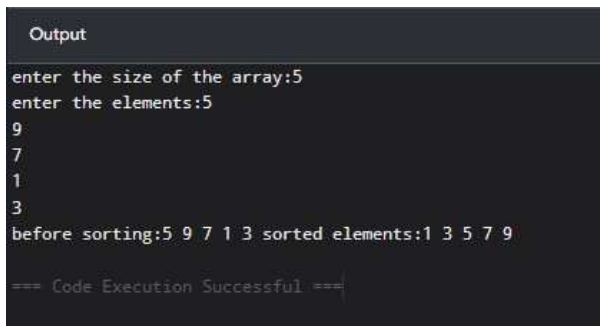
void printelements() {
    for (i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
}

void logic() { for (i = 0; i < size -
1; i++) { for (j = 0; j < size - i -
1; j++) { if (arr[j] > arr[j + 1]) {
temp = arr[j]; arr[j] = arr[j +
1];

arr[j + 1] = temp;
}
```

```
    }  
}  
}  
};  
  
int main() { bubblesort  
    bubble; bubble.getinput();  
    cout << "before sorting:";  
    bubble.printelements();  
    bubble.logic(); cout <<  
    "sorted elements:";  
    bubble.printelements();  
    return 0;  
}
```

OUTPUT:



The screenshot shows the output of the program in a dark-themed terminal window. The text is as follows:

```
Output  
enter the size of the array:5  
enter the elements:5  
9  
7  
1  
3  
before sorting:5 9 7 1 3 sorted elements:1 3 5 7 9  
=== Code Execution Successful ===
```

2.) SELECTION SORT:

PROGRAM:

```
#include <iostream>  
using namespace std;  
class selectionsort{  
public:  
    int arr[50], size, i, j, temp, min; void getinput(){ cout<< "enter the size of the array:"; cin>> size;  
  
    cout<< "enter the elements:";  
    for(i = 0; i < size; i++){  
        cin >> arr[i];  
    }  
}
```

```
}

void printelements(){
for(i = 0; i < size; i++){
cout << arr[i] << " ";
}
}

void logic(){ for(i = 0; i
< size - 1; i++){ min = i;

for(j = i + 1; j < size; j++){
if(arr[j] < arr[min]){
min = j;
}
}

temp = arr[i];
arr[i] = arr[min];
arr[min] = temp;
}
}
};

int main(){

selectionsort selection;

selection.getinput();

cout << "before sorting:";
selection.printelements();
selection.logic();

cout << "\nsorted elements:";
selection.printelements(); return 0;
}
```

OUTPUT:

```
Output
enter the size of the array:5
enter the elements:21
35
10
8
43
before sorting:21 35 10 8 43
sorted elements:8 10 21 35 43
=== Code Execution Successful ===
```

3.) INSERTION SORT:

PROGRAM:

```
#include <iostream>

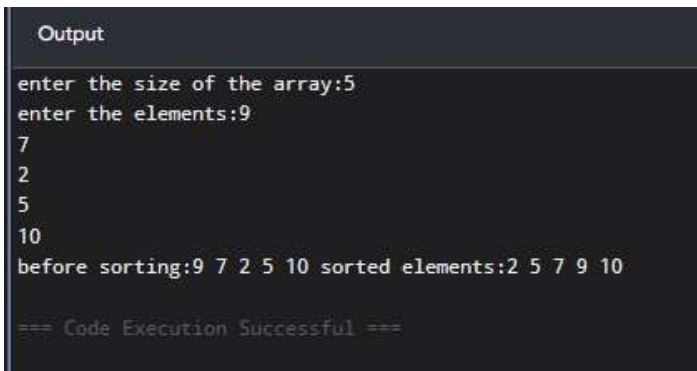
using namespace std;

class insertionsort {
public:
    int arr[100], size, i, j, temp; void
    getinput() { cout << "enter the size of
    the array:"; cin >> size; cout << "enter
    the elements:";
        for (i = 0; i < size; i++) {
            cin >> arr[i];
        }
    }
    void printelements() {
        for (i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
    }
    void logic() { for (i = 1; i
    < size; i++) { temp =
    arr[i];
```

```
j = i - 1;
while (j >= 0 && arr[j] > temp) {
    arr[j + 1] = arr[j];
    j--;
}
arr[j + 1] = temp;
}
}
};
```

```
int main() { insertionsort
insertion;
insertion.getinput(); cout
<< "before sorting:";
insertion.printelements();
insertion.logic(); cout <<
"sorted elements:";
insertion.printelements();
return 0;
}
```

OUTPUT:



```
Output
enter the size of the array:5
enter the elements:9
7
2
5
10
before sorting:9 7 2 5 10 sorted elements:2 5 7 9 10

=== Code Execution Successful ===
```

4.) QUICK SORT:

PROGRAM:

```
#include <stdio.h> int
partition(int arr[], int lb, int ub)
{
    int pivot = arr[lb];
    int start = lb;    int
    end = ub;

    while (start < end)
    {
        while (arr[start] <= pivot)
        {
            start++;
        }
        while (arr[end] > pivot)
        {
            end--;
        }
        if (start < end)
        {
            int temp = arr[start];
            arr[start] = arr[end];    arr[end] =
            temp;
        }
    }

    int temp = arr[lb];
    arr[lb] = arr[end];
    arr[end] = temp;
```

```
return end;
}

void quicksort(int arr[], int lb, int ub)
{
    if (lb < ub)
    {
        int index = partition(arr, lb, ub);
        quicksort(arr, lb, index - 1);
        quicksort(arr, index + 1, ub);
    }
}

int main()
{

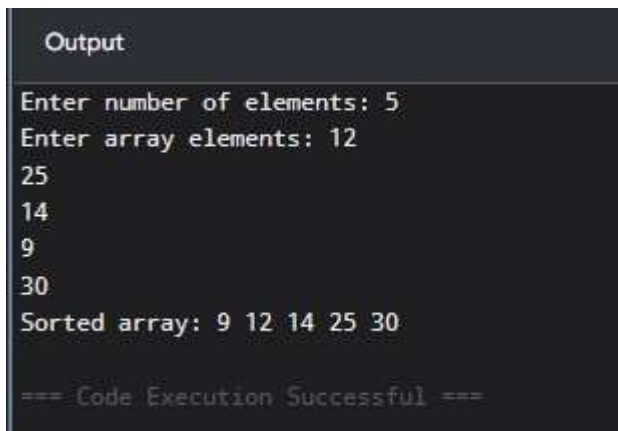
    int arr[100], n;    printf("Enter
number of elements: ");    scanf("%d",
&n);

    printf("Enter array elements: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    quicksort(arr, 0, n - 1);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
```

```
{  
    printf("%d ", arr[i]);  
}  
  
return 0;  
}
```

OUTPUT:



```
Output  
Enter number of elements: 5  
Enter array elements: 12  
25  
14  
9  
30  
Sorted array: 9 12 14 25 30  
  
=== Code Execution Successful ===
```

5.) MERGE SORT

ALGORITHM:

PROGRAM:

```
#include <stdio.h>  
  
void merge(int arr[], int lb, int mid, int  
ub)  
{  
    int i = lb; int j  
    = mid + 1; int  
    k = lb; int  
    b[100];  
  
    while (i <= mid && j <= ub)  
    {  
        if (arr[i] <= arr[j])  
        {  
            b[k] = arr[i];
```



```
i++;
}
else
{
    b[k] =
arr[j]; j++;
}
k++;
}

while (i <= mid)
{
    b[k] = arr[i];
    i++;
    k++;
}

while (j <= ub)
{
    b[k] =
arr[j]; j++;
    k++;
}

for (i = lb; i <= ub; i++)
{
    arr[i] = b[i];
}
}

void mergesort(int arr[], int lb, int ub)
{
    if (lb < ub)
    {
        int mid = (lb + ub) / 2;

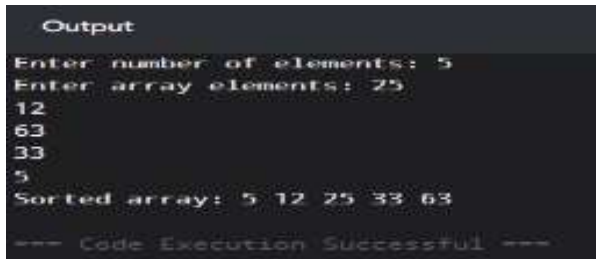
        mergesort(arr, lb, mid);
        mergesort(arr, mid + 1, ub);

        merge(arr, lb, mid, ub);
    }
}

int main()
{
```

```
int arr[100], n;  
  
printf("Enter number of elements: ");  
scanf("%d", &n);  
  
printf("Enter array elements: ");  
for (int i = 0; i < n; i++)  
{  
    scanf("%d", &arr[i]);  
}  
  
mergesort(arr, 0, n - 1);  
  
printf("Sorted array: ");  
for (int i = 0; i < n; i++)  
{  
    printf("%d ", arr[i]);  
}  
  
return 0;  
}
```

OUTPUT:



The screenshot shows the output of a C program. It starts with a title 'Output' in a dark box. The user is prompted to enter the number of elements, which is 5. Then, the user is prompted to enter array elements, which are 12, 63, 33, 5, and 25. The program then displays the sorted array: 5 12 25 33 63. At the bottom, it says '=== Code Execution Successful ==='.